

1. Tổng quan đề tài, ý nghĩa và mục đích

○ Tổng quan đề tài

Đề tài thực hiện một bài toán học máy: **Tối ưu hóa hàm mất mát trong bài toán hồi quy Ridge dự đoán lãi suất khoản vay trên bộ dữ liệu Lending Club.**

Trọng tâm của đề tài là **cực tiểu hóa hàm mất mát** của mô hình thông qua việc tự cài đặt và so sánh các thuật toán tối ưu:

- Gradient Descent (GD)
- Stochastic / Mini-batch Gradient Descent (SGD)
- Accelerated Gradient Descent (Nesterov)
- Newton's Method

Mỗi thuật toán được thử nghiệm với cả **độ dài bước cố định** và **Backtracking Line Search**.

○ Ý nghĩa

Đề tài giúp người học hiểu sâu hành vi thực tế của các thuật toán tối ưu khi áp dụng lên một hàm mất mát lồi và trơn trên dữ liệu lớn. Thông qua thực nghiệm, có thể quan sát rõ sự khác biệt về tốc độ hội tụ, độ ổn định của đường loss và chi phí tính toán giữa các phương pháp bậc nhất và bậc hai.

Mục đích

1. Tự cài đặt các thuật toán tối ưu phổ biến (GD, SGD, Accelerated GD, Newton) kèm theo cơ chế chọn bước nhảy cố định và Backtracking.
2. Chạy thực nghiệm trên dữ liệu lớn để quan sát và phân tích hành vi hội tụ của từng thuật toán (đường loss mượt hay zigzag, số vòng lặp, thời gian chạy...).
3. So sánh hiệu năng giữa các thuật toán khi sử dụng cấu hình tốt nhất của từng phương pháp.
4. Đối chiếu kết quả tự cài đặt với thư viện chuẩn (sklearn.linear_model.Ridge) về giá trị hàm mất mát và thời gian tính toán.

2. Phương pháp thực hiện

○ Bài toán học máy

Dự đoán lãi suất khoản vay bằng Ridge Regression.

Hàm mất mát cần cực tiểu hóa:

$$J(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2 \quad J(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

$$J(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

Hàm này lồi mạnh và khả vi, có gradient cùng Hessian dạng đóng, phù hợp để triển khai đầy đủ các thuật toán yêu cầu.

Các thuật toán được triển khai:

- Gradient Descent (full-batch)
- Mini-batch SGD
- Nesterov Accelerated Gradient
- Newton Method

Mỗi thuật toán đều được thử với độ dài bước cố định và Backtracking Line Search.

Cách đánh giá:

- Theo dõi giá trị hàm mất mát theo số vòng lặp và theo thời gian thực.
- So sánh giữa các thuật toán ở cấu hình tốt nhất.
- So sánh với kết quả của `sklearn.linear_model.Ridge`.

3. Những việc cần làm (Lý thuyết + Thực nghiệm)

Phần lý thuyết cần nắm:

- Phát biểu bài toán Ridge Regression dưới dạng tối ưu hóa hàm mất mát.
- Công thức gradient và Hessian của hàm mất mát.
- Nguyên lý hoạt động của GD, SGD, Nesterov và Newton.
- Cơ chế Backtracking Line Search (điều kiện Armijo).
- Các tính chất kỳ vọng: full-batch cho đường loss mượt, mini-batch tạo độ zigzag, Newton hội tụ nhanh theo số vòng lặp...

Phần thực nghiệm cần làm:

1. Tiền xử lý dữ liệu Lending Club (chọn đặc trưng số, xử lý thiếu, chuẩn hóa) và lấy khoảng 1–1.5 triệu mẫu.
<https://www.kaggle.com/datasets/wordsforthewise/lending-club>
2. Viết hàm tính Loss, Gradient và Hessian.
3. Cài đặt đầy đủ 4 thuật toán với cả fixed step size và Backtracking.
4. Thử nhiều giá trị learning rate và hệ số λ lambda λ , từ đó chọn cấu hình tốt nhất cho từng thuật toán.
5. Vẽ hai biểu đồ bắt buộc:
 - Giá trị hàm mất mát theo số vòng lặp
 - Giá trị hàm mất mát theo thời gian chạy

- So sánh kết quả tự viết với sklearn.Ridge về giá trị loss cuối cùng và thời gian tính toán.
- Phân tích kết quả thực nghiệm để giải thích các hiện tượng quan sát được (độ mượt của đường loss, tốc độ hội tụ, ảnh hưởng của batch size, learning rate...).

Tham khảo: <https://www.kaggle.com/code/faressayah/lending-club-loan-defaulters-prediction>

1. Bài toán gốc: Ridge Regression

Ta có dữ liệu:

- $X \in \mathbb{R}^{n \times d}$: ma trận đặc trưng (n mẫu, d đặc trưng)
- $y \in \mathbb{R}^n$: vector lãi suất thực tế (int_rate)

Mô hình tuyến tính: $\hat{y} = Xw$

Hàm mất mát cần cực tiểu hóa:

$$J(w) = \frac{1}{2n} \|Xw - y\|_2^2 + \frac{\lambda}{2} \|w\|_2^2$$

- Thành phần đầu: Mean Squared Error (chia $2n$ để gradient đẹp)
- Thành phần sau: **L2 regularization** (Ridge), $\lambda > 0$

Tính chất quan trọng:

- $J(w)$ **lồi mạnh** (strongly convex)
- $J(w)$ **khả vi liên tục** (smooth) $\rightarrow$ Gradient Descent và Newton đều hội tụ về điểm cực tiểu duy nhất w^*

2. Gradient và Hessian (bắt buộc phải nhớ)

Gradient (vector đạo hàm bậc 1):

$$\nabla J(w) = \frac{1}{n} X^T (Xw - y) + \lambda w$$

Hessian (ma trận đạo hàm bậc 2):

$$\nabla^2 J(w) = \frac{1}{n} X^T X + \lambda I$$

Hessian **không phụ thuộc vào** w (vì hàm bậc 2). Đây là lý do Newton rất mạnh trên bài toán này.

3. Các thuật toán tối ưu

3.1. Gradient Descent (Full-batch)

Công thức cập nhật:

$$w_{k+1} = w_k - \alpha \nabla J(w_k)$$

- α : learning rate (độ dài bước)
- Mỗi vòng lặp dùng **toàn bộ** n mẫu $\rightarrow$ đường loss rất mượt
- Chi phí mỗi vòng: $O(nd)$

3.2. Mini-batch Stochastic Gradient Descent (SGD)

Thay vì dùng toàn bộ dữ liệu, mỗi lần chỉ lấy 1 mini-batch B (ví dụ 256, 512, 1024 mẫu):

$$\nabla J_B(w) = \frac{1}{|B|} X_B^T (X_B w - y_B) + \lambda w$$

$$w_{k+1} = w_k - \alpha \nabla J_B(w_k)$$

- Đường loss **zigzag** mạnh
- Mỗi vòng rẻ hơn nhiều $\rightarrow$ chạy được nhiều vòng hơn trong cùng thời gian
- Batch size càng nhỏ $\rightarrow$ nhiễu càng lớn, càng khó hội tụ chính xác

3.3. Nesterov Accelerated Gradient (NAG)

Nesterov là phiên bản “có quán tính” của GD. Ý tưởng: nhìn về phía trước rồi mới tính gradient.

Phiên bản chuẩn:

$$\begin{aligned} v_{k+1} &= \beta v_k + \nabla J(w_k + \beta v_k) \\ w_{k+1} &= w_k - \alpha v_{k+1} \end{aligned}$$

Hoặc viết gọn hơn (dạng phổ biến):

$$y_k = w_k + \beta(w_k - w_{k-1})$$

$$w_{k+1} = y_k - \alpha \nabla J(y_k)$$

- β thường lấy khoảng 0.9
- Hội tụ nhanh hơn GD thuần rất nhiều trên hàm lồi mạnh
- Đường loss vẫn khá mượt (vì full-batch)

3.4. Newton's Method

Dùng thông tin bậc 2:

$$w_{k+1} = w_k - \alpha (\nabla^2 J(w_k))^{-1} \nabla J(w_k)$$

Vì Hessian không đổi theo w , ta có thể tính **một lần** rồi dùng lại:

$$H = \frac{1}{n} X^T X + \lambda I$$

Sau đó mỗi vòng chỉ cần giải hệ tuyến tính $Hd = \nabla J(w)$ (hoặc dùng np.linalg.solve).

- Hội tụ **cực nhanh** theo số vòng lặp (thường chỉ cần 5–15 vòng)
- Mỗi vòng rất đắt (tính Hessian + giải hệ $d \times d$)
- Trên dữ liệu lớn ($n \approx 1 - 1.5$ triệu) thường chậm hơn GD/Nesterov về **thời gian thực**

4. Fixed step size vs Backtracking Line Search

Fixed step size

Chọn sẵn α cố định (thử nhiều giá trị rồi chọn cái tốt nhất).

Nhược điểm: quá lớn $\rightarrow$ phân kỳ, quá nhỏ $\rightarrow$ hội tụ chậm.

Backtracking Line Search (Armijo)

Ý tưởng: bắt đầu từ α lớn, giảm dần cho đến khi thỏa điều kiện giảm đủ.

Điều kiện Armijo:

$$J(w - \alpha \nabla J(w)) \leq J(w) - c\alpha \|\nabla J(w)\|^2$$

Thường lấy $c = 10^{-4}$, và hệ số giảm $\rho = 0.5$.

Thuật toán Backtracking:

text

Copy

$\alpha = \alpha_0$ (ví dụ 1.0)

while $J(w - \alpha g) > J(w) - c \cdot \alpha \cdot \|g\|^2$:

$\alpha = \rho \cdot \alpha$

Ưu điểm:

- Tự động tìm bước phù hợp
- An toàn hơn fixed step
- Dùng được cho cả GD, Nesterov, Newton

5. Những tính chất kỳ vọng bạn sẽ thấy trong thực nghiệm

Thuật toán	Đường loss	Số vòng lặp đến hội tụ	Thời gian thực (dữ liệu lớn)	Ghi chú
Full GD	Rất mượt	Nhiều	Trung bình	Đơn giản, ổn định
Mini-batch SGD	Zigzag mạnh	Rất nhiều	Thường nhanh nhất	Batch size quan trọng
Nesterov	Mượt + giảm nhanh	Ít hơn GD	Thường tốt nhất	Nên thử kỹ
Newton	Giảm cực nhanh	Rất ít (5–20)	Chậm (do Hessian)	Chỉ mạnh khi d nhỏ

6. Những việc bạn cần làm trong code (checklist)

- Viết 3 hàm:
 - $\text{loss}(w, X, y, \text{lam})$
 - $\text{grad}(w, X, y, \text{lam})$
 - $\text{hessian}(X, \text{lam})$ (chỉ cần 1 lần)
- Cài 4 optimizer, mỗi cái có 2 phiên bản:
 - Fixed step size
 - Backtracking
- Chạy thử nhiều learning rate / batch size / $\lambda \rightarrow$ chọn cấu hình tốt nhất của từng thuật toán.
- Vẽ 2 biểu đồ bắt buộc:
 - Loss theo số vòng lặp
 - Loss theo thời gian chạy (dùng `time.time()`)

5. So sánh loss cuối cùng và thời gian với `sklearn.linear_model.Ridge`

7. Lời khuyên thực chiến

- Bắt đầu với **Full GD + Backtracking** trước (dễ debug nhất).
- λ thử khoảng $10^{-3} \rightarrow 10$ (log scale).
- Learning rate fixed: thử $10^{-3}, 10^{-2}, 10^{-1}, 1, 10 \dots$
- Batch size: 256, 512, 1024, 2048.
- Chuẩn hóa dữ liệu (StandardScaler) trước khi train — rất quan trọng.
- Khi so sánh thời gian, phải chạy trên cùng máy, cùng số sample.